

AFRILANG Stdlib

std/c/sigfft

Bibliothèque AFRILANG `std/c/sigfft` (niveau complex, catégorie complex). Elle expose 6 fonction(s) : dft, magnitude, po

```
`import "std/c/sigfft" · `use sigfft`
```

- `dft(signal list number) ? list number`
- `magnitude(signal list number) ? list number`
- `powerSpectrum(signal list number) ? list number`
- `dominantFreq(signal list number) ? number`
- `inverseDft(spectrum list number, n number) ? list number`
- `bandEnergy(signal list number, lo number, hi number) ? number`

Category: complex | Tier: complex | Functions: 6

FRANCAIS

1. Vue d'ensemble

Bibliothèque AFRILANG `std/c/sigfft` (niveau complex, catégorie complex). Elle expose 6 fonction(s) : dft, magnitude, po

```
`import "std/c/sigfft"` . `use sigfft`  
- `dft(signal list number) ? list number`  
- `magnitude(signal list number) ? list number`  
- `powerSpectrum(signal list number) ? list number`  
- `dominantFreq(signal list number) ? number`  
- `inverseDft(spectrum list number, n number) ? list number`  
- `bandEnergy(signal list number, lo number, hi number) ? number`
```

2. Installation & import

Ajoutez le module à votre programme :

```
import "std/c/sigfft"  
use sigfft
```

Niveau : complex · Catégorie : Modules complex (c/)
Domaines avancés ? graphes, NLP, réseaux complexes.

3. Référence API

```
? dft(signal list number) ? list  
? magnitude(signal list number) ? list  
? powerSpectrum(signal list number) ? list  
? dominantFreq(signal list number) ? number  
? inverseDft(spectrum list number, n number) ? list  
? bandEnergy(signal list number, lo number, hi number) ? number
```

4. Exemples d'utilisation

```
import "std/c/sigfft"  
use sigfft  
  
create result = dft(/* args */)   
say result  
  
create result = magnitude(/* args */)   
say result  
  
create result = powerSpectrum(/* args */)   
say result  
  
create result = dominantFreq(/* args */)   
say result  
  
create result = inverseDft(/* args */)   
say result  
  
create result = bandEnergy(/* args */)   
say result
```

5. Bonnes pratiques

```
? Préférez `import "std/c/sigfft"` en tête de fichier.  
? Lancez `afrilang check` avant de livrer.  
? Combinez avec les modules stdlib de la même catégorie.  
? Voir /stdlib/ pour le source live et les démos playground.  
? Signalez les problèmes sur GitHub si une export manque.
```

6. Annexe ? source

```
module sigfft  
  
function dft(signal list number) returns list number  
end
```

```
function magnitude(signal list number) returns list number
end

function powerSpectrum(signal list number) returns list number
end

function dominantFreq(signal list number) returns number
end

function inverseDft(spectrum list number, n number) returns list number
end

function bandEnergy(signal list number, lo number, hi number) returns number
end
end
```

ENGLISH

1. Overview

AFRILANG library ``std/c/sigfft`` (tier complex, category complex). Exports 6 function(s): `dft`, `magnitude`, `powerSpectrum`,

```
`import "std/c/sigfft"` . `use sigfft`  
- `dft(signal list number) ? list number`  
- `magnitude(signal list number) ? list number`  
- `powerSpectrum(signal list number) ? list number`  
- `dominantFreq(signal list number) ? number`  
- `inverseDft(spectrum list number, n number) ? list number`  
- `bandEnergy(signal list number, lo number, hi number) ? number`
```

2. Installation & import

Add the module to your program:

```
import "std/c/sigfft"  
use sigfft
```

Tier: complex · Category: Complex modules (c/)
Advanced domains ? graphs, NLP, complex networks.

3. API reference

```
? dft(signal list number) ? list  
? magnitude(signal list number) ? list  
? powerSpectrum(signal list number) ? list  
? dominantFreq(signal list number) ? number  
? inverseDft(spectrum list number, n number) ? list  
? bandEnergy(signal list number, lo number, hi number) ? number
```

4. Usage examples

```
import "std/c/sigfft"  
use sigfft  
  
create result = dft(/* args */)   
say result  
  
create result = magnitude(/* args */)   
say result  
  
create result = powerSpectrum(/* args */)   
say result  
  
create result = dominantFreq(/* args */)   
say result  
  
create result = inverseDft(/* args */)   
say result  
  
create result = bandEnergy(/* args */)   
say result
```

5. Best practices

```
? Prefer `import "std/c/sigfft"` at the top of your file.  
? Run `afrilang check` before shipping.  
? Combine with related stdlib modules from the same category.  
? See /stdlib/ for the live source and playground demos.  
? Report issues on GitHub if an export is missing or undocumented.
```

6. Appendix ? source

```
module sigfft  
  
function dft(signal list number) returns list number  
end
```

```
function magnitude(signal list number) returns list number
end

function powerSpectrum(signal list number) returns list number
end

function dominantFreq(signal list number) returns number
end

function inverseDft(spectrum list number, n number) returns list number
end

function bandEnergy(signal list number, lo number, hi number) returns number
end
end
```